

시간 복잡도 / 공간 복잡도

👤 소유자	📁 암암코딩
🏷 태그	
🕒 생성 일시	@2024년 5월 19일 오후 6:26

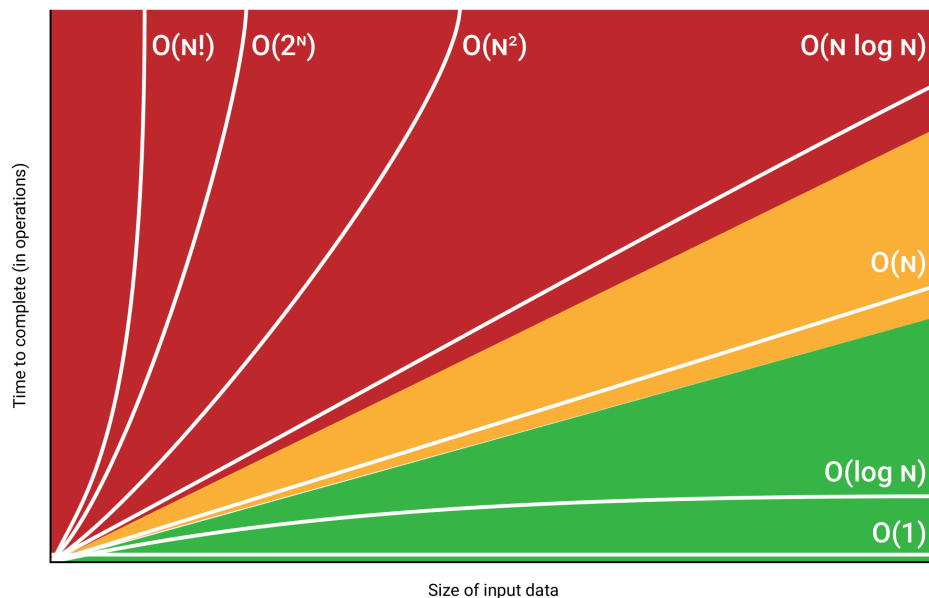
시간 복잡도(Time Complexity) 및 공간 복잡도(Space Complexity)

알고리즘을 평가할 때 시간 복잡도와 공간 복잡도를 사용합니다.

- 시간 복잡도: 알고리즘의 수행시간을 평가
- 공간 복잡도: 알고리즘 수행에 필요한 메모리 양을 평가

시간 복잡도와 공간 복잡도는 주로 점근적 표기법 중 빅오 표기법을 이용하여 나타냅니다.

이유는 최악의 경우에도 해당 알고리즘이 어떤 성능을 낼지 가능해볼 수 있기 때문입니다.



시간복잡도를 알아야 하는 이유

시간복잡도와 공간복잡도를 서로 조율 할 수 있다.

기본적으로 메모리 공간을 많이쓰면 속도가 빨라지고 적게쓰면 연산이 많아진다.

이런 조율을 위해서 우리 프로그램에 시간복잡도와 공간복잡도 계산을 할 수 있어야 한다.

O(n)

입력받은 수만큼 또는 데이터의 크기만큼 반복하는 경우

```
for (size_t i = 0; i < length; i++)  
{  
    //  
}
```

O(3n) 으로 표기해야 될것 같지만 간략하게 O(n)으로 표기한다.

빅오 표기법은 반복문을 기준으로 성능을 간략하게 표현한다.

```
for (size_t i = 0; i < length; i++)  
{  
  
}  
  
for (size_t i = 0; i < length; i++)  
{  
  
}  
  
for (size_t i = 0; i < length; i++)  
{  
  
}
```

O(1)

입력값과 상관없이 일정한 속도를 보인다면 해당 시간복잡도는 O(1)로 표현다.

[상수] 횟수만큼 반복이라면 → O(1)

```
for (size_t i = 0; i < 6; i++)  
{  
  
}  
  
for (size_t i = 0; i < 10000; i++)  
{  
  
}
```

빅오표기법에서는 통상적으로 O(n)보다 O(1) 더 빠른 알고리즘이라고 판단

아래와 같은 코드에서는 반복횟수가 1000 * 1000000 번으로

아주 느린 프로그램이지만, 통상적으로는 O(n) 더 느리다고 판단.

굉장히 반복횟수가 많지만 n값과 상관없이 일정하게 항상 반복하기때문에

아래 코드의 성능은 O(1)이다.

```
for (size_t i = 0; i < 10000; i++)  
{  
    for (size_t j = 0; j < 1000000; j++)  
    {  
  
    }  
}
```

$O(n^2)$

이중 반복문이 대표적인 예시이다.

```
for (size_t i = 0; i < n; i++)
{
    for (size_t j = 0; j < m; j++)
    {
        // ...
    }
}
```

```
for (size_t i = 0; i < n; i++)
{
    for (size_t j = 0; j < m; j++)
    {
        // ...
    }

    for (size_t j = 0; j < m; j++)
    {
        // ...
    }
}
```

$O(\log n)$

주로 2진트리에서 이러한 성능을 나오게 된다.

알고리즘 문제풀이 분야에서 $\log N$ 의 기본 지수값은 2이다.

정확히는 $O(\log 2N)$ 여기서 지수 2를 생략해서 표현하고 오더 로그엔이라고 부른다.

$O(n)$ 에서 n 값이 100,000일때 반복 횟수는 100,000번

$O(n^2)$ 에서 n 값이 100,000일때 반복 횟수는 100,000, 00000번

$O(\log N)$ 에서 n 값이 100,000일때 반복 횟수는 $2^{15.61} \dots$ 16번

$O(n \log n)$

$O(\log N)$ 을 n 번 반복하는 알고리즘 이다.

$O(\log N)$ 는 $O(1)$ 에 가까운 알고리즘이기 때문에

$O(n \log n)$ 은 $O(n)$ 과 비슷한 성능을 보인다고 생각하면 된다.



시간을 체크하는 방법

```
#include <iostream>
#include <ctime>
using namespace std;
```

```
int main() {
    clock_t start, finish;
    double duration;

    start = clock();

    /*실행 시간을 측정하고 싶은 코드*/

    finish = clock();

    duration = (double)(finish - start) / CLOCKS_PER_SEC;
    cout << duration << "초" << endl;

    return 0;
}
```

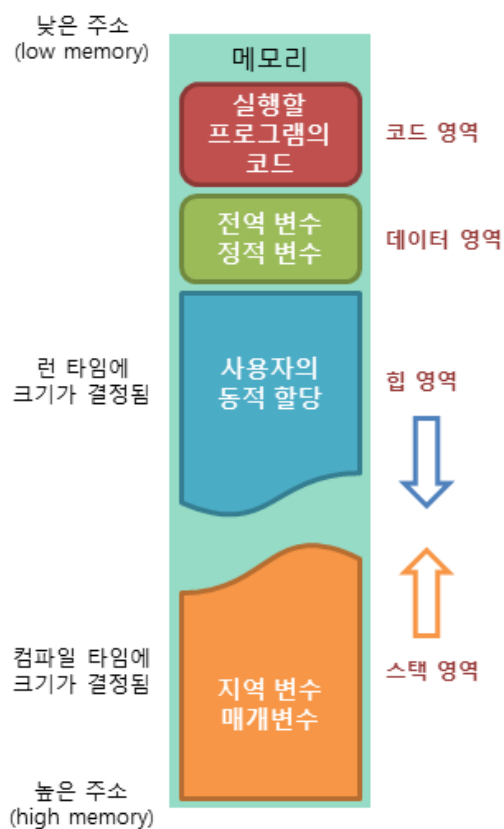
코드안에 어떤 소스코드가 있는지에 따라 걸리는 시간은 천차만별이다.

또한 멀티쓰레드 기준이라면 속도측정이 어려운경우도 많다.

그래도 대략적인 기준을 잡아보자면 1억반복당 1초걸린다고 생각하면 된다.(단일쓰레드)

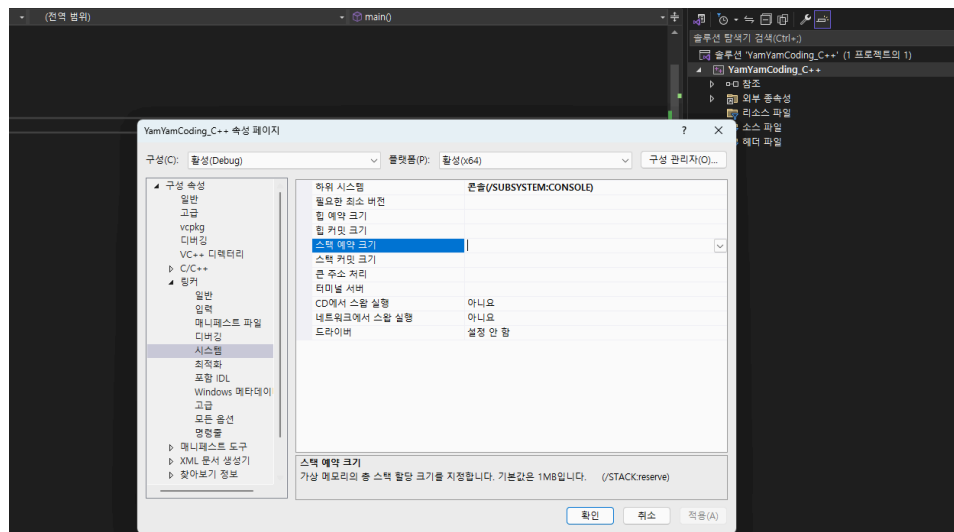
공간복잡도 : 메모리 사용량

메모리를 얼마나 사용하고 있는지를 계산 할 수 있어야 한다.



스택의 크기는 기본적으로 1MB 설정되어있다.

필요에 따라서 비주얼스튜디오에서는 옵션으로 바꿔줄 수 있다.

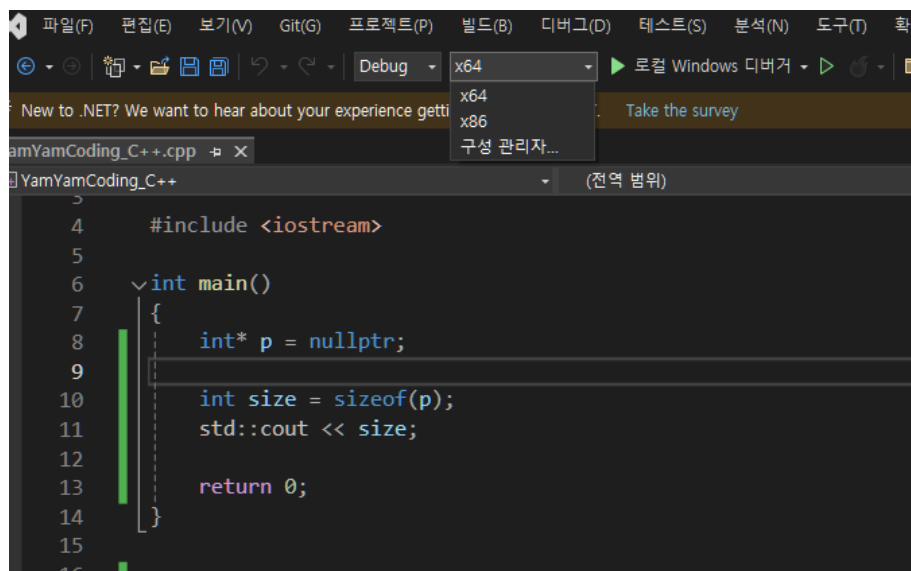


32bit/64bit 컴파일

x64는 64비트 컴파일러이고 x86 32비트로 컴파일 이 된다.

각각의 데이터 연산의 크기가 다르기때문에 데이터의 크기도 달라지고 사용가능한 메모리의 양도 다르다.

하지만 최근에는 (2024.05)기준 32비트는 개발되지 않는다고 보된다.



메모리 사용량을 계산하고 싶다면 **sizeof() 연산자**를 사용해서 계산 해보면 된다.

구조체/ 클래스에서 바이트 패딩

패딩이란 구조체나 클래스를 실제로 메모리에 올릴 때, 성능 향상을 위해 추가적인 메모리를 할당하여 끼워 넣는 것을 의미한다.

패딩을 하는 이유(인터넷 펌)

메모리에서 CPU 레지스터로 한번에 읽어오는(fetch) 데이터의 크기 때문에 패딩이 일어납니다. 32비트 머신에서는 4바이트 씩이고 64비트 머신에서는 8바이트 씩이겠지요.

이렇게 선언을 하면 메모리 맵상의 어딘가에 영역이 잡히겠죠

그런데 패딩이 없다고 가정하면,

- -----

|a|bb|cccc|a|bb|cccc|a|bb|cccc|a|bb|cccc|...

- -----

이런식으로 잡히겠죠...

fetch는 4바이트 단위로 이루어지니깐 만약 첫번째 c를 접근하기 위해서는

2번의 fetch가 이루어져야 합니다.

0번째 바이트부터 3번째 바이트까지 4바이트를 읽어서 그중에 3번째 바이트를 먼저 취하고,

4번째 바이트부터 7번째 바이트까지 4바이트 읽어서 그중에 4~6번째 바이트를 취해서,

둘을 합해 최종 c의 값을 만드는 결과가 생기지요~

때문에 컴파일러가 패딩을 넣는 겁니다.

한줄요약 : 구조체/클래스 패딩은 구조체 안의 가장 큰 자료형의 크기로 정해진다.

만약 구조체 안에 구조체가 들어가 있다면, 그 구조체 안에서 가장 큰 자료형의 크기로 정해진다.

만약 **설정이 8byte 패딩**으로 되어 있는데 구조체가

```
class C
{
    short a;
    int b;
    int c;
};
```

이와 같이 구성되어 있다면, **short-int를 하나로 묶어서 6바이트 + 2바이트 패딩(총 8바이트)**

int + 패딩 4바이트 (총 8바이트)

식으로 묶는다. (단, short는 2bytes, int는 4bytes일 때의 풀이이다)

만약 설정이 4bytes 패딩이라면,

short a; //4bytes

int b; //4bytes

int c; //4bytes

총 12바이트가 된다.

이름	값
sizeof(C)	12

예시

```
struct Name //2byte
{
    char name[2];
};
```

- → 답은 2이다. 왜냐하면 패딩은 레지스터 단위 접근을 쉽게 하기 위해서 만드는 것인데 char[2] 는 2바이트이며, 다음에 접근할만한 요소가 없기 때문에 2만 차지해도 되기 때문이다(만약 int형 변수가 뒤에 하나 더 선언되어 있다면 총 용량은 2 + 2(패딩) + 4였을 것이다.)

```
struct Point //8 byte
{
    //8
    short point_x;
    int point_y;
};

**//short + 2byte / int**

struct Player // 20 byte
{
    char rank;
    short hp;
    short mp;
    Point point;
    Name name;
};
```

Point의 **제일 큰 타입이 int**이므로, 4byte 기준으로 패딩이 잡힘.

char + short + 1byte 패딩 / short + 2byte 패딩 / 8byte(point) / 4byte (name)

mp와 point는 따로 용량을 잡는다.

```
struct long_Point //16
{
    double point_x;
    int point_y;
};
```

double = 8bytes이므로 총 메모리 용량은 8+8 = 16bytes

소켓 프로그래밍 중 패킷 구조체를 구현할 때 **#pragma pack(push, 1)**을 쓰는 이유도

패딩에 있다.

패킷을 받은 쪽은 패킷을 읽어서 값을 복구해야 하는데(캐스팅을 통해), 패딩이 들어가 있으면 잘못된 크기를 가지고 캐스팅을 하게 되므로 잘못된 값이 복구되기 때문이다.



문제 1번

민철이가 짠 네 개의 소스코드를 읽어보고, 시간복잡도를 기준으로 채점 해 주세요.

빠른 알고리즘일수록 점수가 높습니다.

문제 번호를 입력하면, 그 문제의 점수를 출력 해 주세요.

$O(N^3)$ = 1 점

$O(N^2)$ = 2 점

$O(N \log N)$ = 10 점

$O(N)$ = 11점

$O(\log N)$ = 20점

$O(1)$ = 21점

1번 소스코드

```
#include <iostream>
using namespace std;

int main()
{
    for (int i = 0; i < 10000; i++) {
        cout << "#";
    }

    return 0;
}
```

2번 소스코드

```
#include <iostream>
using namespace std;

int main()
{
    int n;
    cin >> n;

    for (int y = 0; y < n; y++) {
        for (int x = 0; x <= y; x++) {
            cout << "##";
        }
    }

    return 0;
}
```

3번 소스코드

```
#include <iostream>
using namespace std;

int n;

void abc()
```



```

{
    for (int i = 0; i < n; i++) {
        cout << "#";
    }
}

int main()
{
    cin >> n;

    for (int y = 0; y < n; y++) {
        abc();
        abc();
        abc();
    }

    return 0;
}

```

4번 소스코드

```

#include <iostream>
using namespace std;

int main()
{
    cin >> n;

    for (int y = 0; y < n; y++) {

        for (int x = 0; x < 5; x++) {

            for (int z = 0; z < n; z++) {
                cout << "#";
            }
        }
    }

    return 0;
}

```

입력 예제

1

출력 결과

21



문제 2번

숫자 n 과 n 개의 숫자를 입력 받아주세요. ($4 \leq n \leq 100,000$)

만약 9를 입력받고, ($n = 9$)

7 2 4 3 2 1 1 9 2를 입력받았다면

아래와 같이 숫자가 채워집니다.

7	2	4	3	2	1	1	9	2
---	---	---	---	---	---	---	---	---

배열에서 연속 되어있는 네 칸의 합을 구했을 때,

가장 작은 값이 몇 인지 출력 해 주세요

위 예제에서는 최소 합은 7 입니다. ($3 + 2 + 1 + 1$)

- **Sliding Window** 알고리즘을 써서 풀어주세요

입력 예제

9

7 2 4 3 2 1 1 9 2

출력 결과

7



문제 3번

아래 소스코드가 각각 메모리를 얼마나 차지하는지 계산 후,
답을 출력하는 프로그램을 만드세요
만약 1번 소스코드 정답이 1000 Byte이고,
만약 2번 소스코드 정답이 2000 Byte라면
if (input == 1) cout << 1000;
if (input == 2) cout << 2000;
위와 같은 방식으로 프로그램을 작성해서 제출하시면 됩니다.

<1번문제>

```
#include <iostream>
using namespace std;
int data[10];
int main()
{
    return 0;
}
```

<2번문제>

```
#include <iostream>
using namespace std;
double data[3];
char vect[10];
int dt[10];
int main()
{
    return 0;
}
```

<3번문제> padding 포함

```
#include <iostream>
using namespace std;
struct Node
{
    int x;
    char t;
};
Node vect[100];
int main()
{
    return 0;
}
```

<4번문제> 64bit 컴파일 했을 때, padding 포함

```
#include <iostream>
using namespace std;
struct Node { int x; char *next; };
Node vect;
int main() { return 0; }
```

입력 예제

1

출력 결과

40



문제 4번

아래와 같이 다섯 글자로 이루어진 5개 문자열을 **string배열**에 입력받습니다.

먼저 세로 1번 Line (노랑)과 세로 3번 Line (빨강)을 교체합니다.

A	B	C	D	E
Q	W	E	R	T
Z	X	C	V	B
O	P	E	R	A
M	O	P	A	M

A	D	C	B	E
Q	R	E	W	T
Z	V	C	X	B
O	R	E	P	A
M	A	P	O	M

이제 한 줄씩 비교하여 MAPOM 이라는 문자열이 존재하는지 찾으면 됩니다.

char배열에 먼저 입력받고, string으로 MAPOM을 검색 해 주세요.

만약 다섯줄 중 MAPOM이 발견된다면 yes, 아니면 no를 출력 해 주세요.

입력 예제

ABCDE

QWERT

ZXCVB

OPERA

MOPAM

출력 결과

yes



문제 5번

숫자 N과 N개의 문자열을 입력받습니다.

문자열을 배열 [0] ~ [N-1] index에 저장하면 됩니다.

이제, N개의 문자열 중 2개만 선택하여 새로운 문자열을 만들어보려고 합니다.

첫 번째로 선택한 문자열은 반드시 두 번째로 선택한 문자열보다 더 왼쪽에 있는 문자열이어야 합니다.

(즉, [첫 번째 문자열 index] < [두 번째 문자열 index])

2개의 문자열을 선택하여

"HITSMUSIC" 이라는 문장을 만들 수 있는 방법이 총 몇 가지인지 출력 해 주세요.

1. ex) 다음과 같이 7개의 문장을 입력받았다면

HITS	HI	HITSM	MUSIC	SMU	SIC	USIC
------	----	-------	-------	-----	-----	------

만들 수 있는 총 경우는

[0] + [3] = HITSMUSIC

[2] + [6] = HITSMUSIC

답은 2 입니다.

[힌트]

2중 for문

입력 예제

7

HITS HI HITSM MUSIC SMU SIC USIC

출력 결과

2



문제 6번

사유지에 버스를 주차하려고 합니다.

만약 이 버스가 차지하는 칸이 세 칸이고,

0~2번 칸에 주차를 한다면 주차요금은 $(1 + 2 + 3 = 6)$ 6만원 입니다.

그런데 5~7번 칸에 주차를 한다면 주차요금은 $(1 + 0 + 1 = 2)$ 2만원으로 최저요금입니다.



도로별 주차요금 (9칸)을 하드코딩 해 주세요.

버스가 차지하는 칸 수를 입력받았을 때,

최저요금은 얼마인지 찾아서 출력 해 주세요.

(*Sliding Window 알고리즘으로 풀어주세요)

입력 예제

3

출력 결과

2



문제 7번

n개의 문자열을 입력 받아주세요. (최대 20개)

길이 순으로(오름차순), 그리고 같은 길이 일때는 사전순(오름차순)으로 정렬하여 출력 해 주세요.

만약 아래와 같이 7개 문자열을 입력받았다면,

ABC	B	TTS	FRIENDS	TRUE	LOVE	MORETIM
-----	---	-----	---------	------	------	---------

길이 순 + 사전순으로 정렬하면 다음과 같습니다.

B

ABC

TTS

LOVE

TRUE

FRIENDS

MORETIM

[힌트]

```
string a = "ABC";
```

```
string b = "BHC";
```

```
if (a > b) cout << "a가 사전순으로 더 뒤에 있음";
```

```
else cout << "b가 사전순으로 더 뒤에 있음";
```

입력 예제

7

ABC

B

TTS

FRIENDS

TRUE

LOVE

MORETIM

출력 결과

B

ABC

TTS

LOVE

TRUE

FRIENDS

MORETIM



문제 8번

숫자 2개(P, N)을 입력 받습니다.

P는 숫자 반죽입니다. 숫자 반죽을 불에 구울수록 2배로 커지고,
다 익으면 뒤집는 동작을 반복하여 부침개를 완성하곤 합니다.

1. 숫자에 2를 곱합니다. ex) 123 → 246

2. 숫자를 뒤집습니다. ex) 246 → 642

위와 같은 행동을 N번 반복한 후, 만들어지는 숫자를 출력 해 주세요.

만약 123 4를 입력받았다면,

1회 : $123 \times 2 = 246 \rightarrow 642$

2회 : $642 \times 2 = 1284 \rightarrow 4821$

3회 : $4821 \times 2 = 9642 \rightarrow 2469$

4회 : $2469 \times 2 = 4938 \rightarrow 8394$

정답은 8394 입니다.

$3926 \times 2 = 7852 \rightarrow 2587$

$2587 \times 2 = 5174 \rightarrow 4715$

$4715 \times 2 = 9430 \rightarrow 349$

$349 \times 2 = 698 \rightarrow 896$

[힌트] 문자열을 숫자로 / 숫자를 문자열로 변경하는 방법

```
int num = 10;
```

```
string str = to_string(num);
```

```
num = stoi(str);
```

입력 예제

123

4

출력 결과

8394



문제 9번



할아버지가 가지고 계신 빌딩들의 일부를 내게 물려주신다고 합니다.

부담스러워 극구 거절했지만, 결국 받아들이기로 했습니다.

빌딩의 startIndex와 endIndex를 할아버지께 말씀드리면 그 빌딩들을 받을 수 있습니다.

최대 수익을 얻을 수 있게끔 startIndex와 endIndex를 구해봅시다.

[힌트]

sliding window로 맨 처음부터 한칸씩 영역(window)을 키웁니다.

영역(window)에 해당하는 sum값을 구하면서, 지속적으로 max값을 갱신합니다.

만약 sum이 음수가 되면, 영역(window)을 새롭게 시작합니다.

[입력]

빌딩의 개수가 먼저 입력이되고,

빌딩마다 수익률이 입력됩니다.

[출력]

최대 수익이 되는 빌딩의 startIndex와 endIndex를 구해주세요.

입력 예제

14

1 2 3 2 -3 0 1 -8 3 2 3 -1 2 -3

출력 결과

8 12